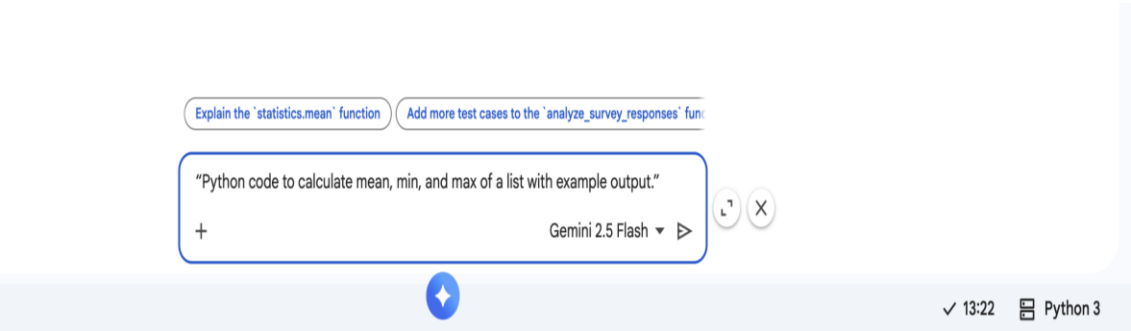
AI Assisted Coding

B.GEETHANJALI
(2303A51005)
BATCH-01

Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab)and Cursor AI.

TASK1: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

Prompt:



Code:

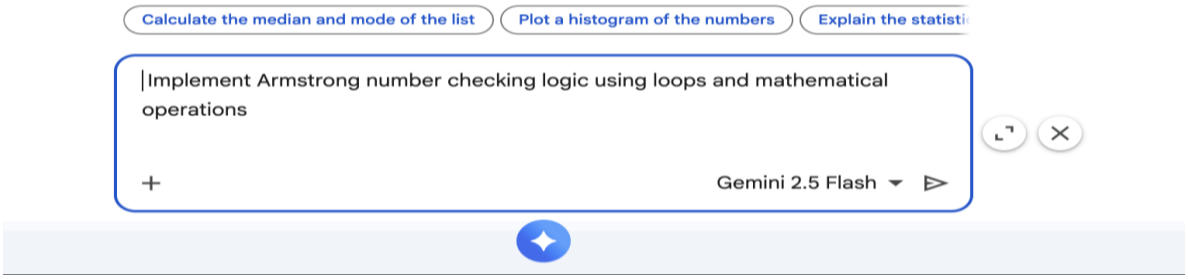
```python
import statistics

# Example list of numbers
numbers = [10, 20, 30, 40, 50, 15, 25, 35, 45]

# Calculate the mean
mean_value = statistics.mean(numbers)

# Calculate the minimum value
min_value = min(numbers)

# Calculate the maximum value
max_value = max(numbers)

# Display the results
print(f"Original list: {numbers}")
print(f"Mean: {mean_value}")
print(f"Minimum: {min_value}")
print(f"Maximum: {max_value}")
```

Output:

```
Original list: [10, 20, 30, 40, 50, 15, 25, 35, 45]
Mean: 30
Minimum: 10
Maximum: 50
```

TASK2: Generate an Armstrong number checker using Gemini and GitHub Copilot.

Prompt:



Code:

```python
# Function to check if a number is an Armstrong number
def is_armstrong_number(num):
    # Convert number to string to easily get the number of digits
    num_str = str(num)
    n_digits = len(num_str)

    sum_of_powers = 0
    temp_num = num

    # Calculate the sum of the nth powers of its digits
    while temp_num > 0:
        digit = temp_num % 10
        sum_of_powers += digit ** n_digits
        temp_num //= 10

    # Check if the sum is equal to the original number
    return sum_of_powers == num

# Get input from the user
try:
    number_to_check = int(input("Enter a positive integer: "))
    if number_to_check < 0:
        print("Please enter a positive integer.")
    elif is_armstrong_number(number_to_check):
        print(f"{number_to_check} is an Armstrong number.")
    else:
        print(f"{number_to_check} is not an Armstrong number.")
except ValueError:
    print("Invalid input. Please enter an integer.")
```

Output:

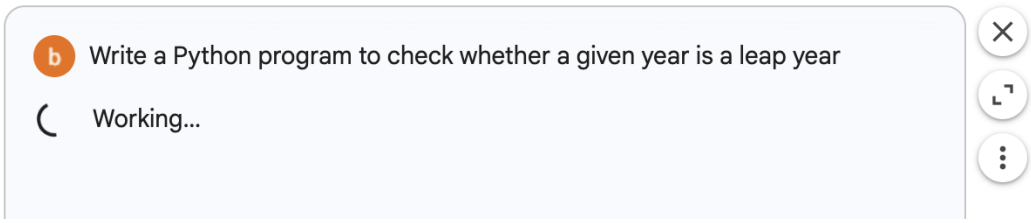
```
Enter a positive integer: 7
7 is an Armstrong number.
```

Side-by-side comparison table:

Aspect	Google Gemini	GitHub Copilot
Code Style	Simple and well-commented	Short and logic-focused
Readability	Very easy for beginners	Moderate, more technical
Logic Approach	Uses string-based digit handling	Uses mathematical operations
Explanation	Provides clear explanations	Minimal explanation
Best Use	Learning and understanding	Fast and efficient coding

TASK3: Use Cursor AI to generate a Python program that checks whether a given is a leap year.

Prompt:



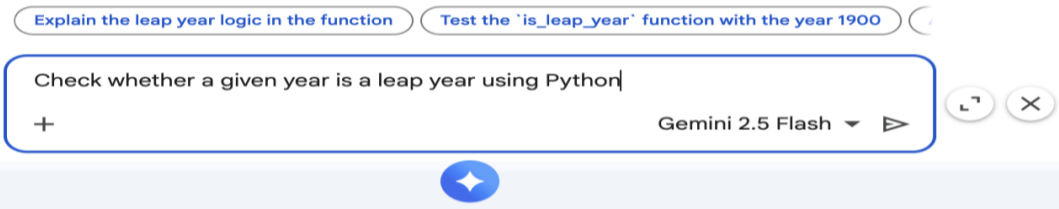
CODE1:

```python
def is_leap_year(year):
    """
    Checks if a given year is a leap year.
    A leap year occurs every four years, except for years that are divisible by 100
    but not by 400.
    """
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
# Example usage:
year_to_check = 2024
if is_leap_year(year_to_check):
    print(f"{year_to_check} is a leap year.")
else:
    print(f"{year_to_check} is not a leap year.")
year_to_check = 2000
if is_leap_year(year_to_check):
    print(f"{year_to_check} is a leap year.")
else:
    print(f"{year_to_check} is not a leap year.")

year_to_check = 1900
if is_leap_year(year_to_check):
    print(f"{year_to_check} is a leap year.")
else:
    print(f"{year_to_check} is not a leap year.")

year_to_check = 2023
if is_leap_year(year_to_check):
    print(f"{year_to_check} is a leap year.")
else:
    print(f"{year_to_check} is not a leap year.")
```

Output:

```
2024 is a leap year.
2000 is a leap year.
1900 is not a leap year.
2023 is not a leap year.
```

Prompt:



CODE2:

```python
def is_leap_year(year):
    """
    Checks if a given year is a leap year.

    A leap year is divisible by 4, except for years divisible by 100 but not by 400.
    """
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False

# Using the year_to_check variable from the kernel state
if is_leap_year(year_to_check):
    print(f"{year_to_check} is a leap year.")
else:
    print(f"{year_to_check} is not a leap year.")
```

Output:

```
2023 is not a leap year.
```

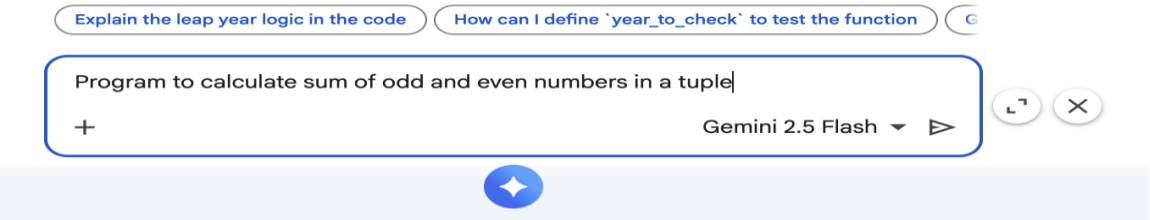
Brief Comparison: A leap year occurs every four years, except for years that are divisible by 100 but not by 400. Here's how to determine if a year is a leap year:

1. If the year is evenly divisible by 4, go to step 2.
2. If the year is evenly divisible by 100, go to step 3.
3. If the year is evenly divisible by 400, it is a leap year. Otherwise, it is not a leap year.

Otherwise, if the year is divisible by 4 but not by 100, it is a leap year. If it's not divisible by 4, it's not a leap year.

TASK4: Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

Prompt: Original code



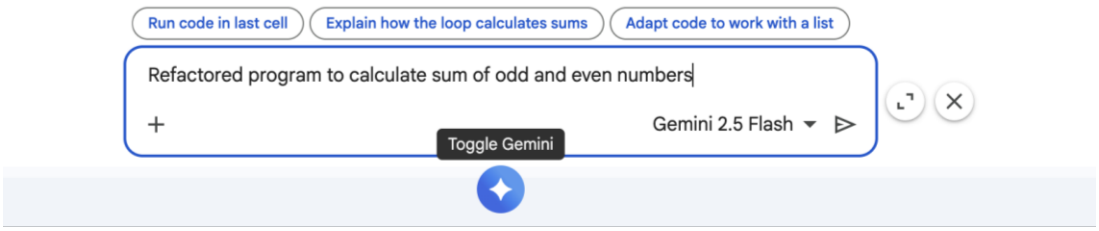
Code:

```
# Define a sample tuple of numbers
numbers_tuple = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

# Initialize sums for odd and even numbers
sum_odd = 0
sum_even = 0

# Iterate through the tuple and calculate sums
for number in numbers_tuple:
    if number % 2 == 0:
        sum_even += number
    else:
        sum_odd += number

# Print the results
print(f"Original tuple: {numbers_tuple}")
print(f"Sum of even numbers: {sum_even}")
print(f"Sum of odd numbers: {sum_odd}")
```

Output:

```
Original tuple: (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
Sum of even numbers: 30
Sum of odd numbers: 25
```

Prompt: Refactored Code



Code:

```python
numbers = [10, 20, 30, 40, 50, 15, 25, 35, 45]

sum_even = 0
sum_odd = 0

for num in numbers:
    if num % 2 == 0:
        sum_even += num
    else:
        sum_odd += num

print(f"Original list: {numbers}")
print(f"Sum of even numbers: {sum_even}")
print(f"Sum of odd numbers: {sum_odd}")
```

Output:

```
Original list: [10, 20, 30, 40, 50, 15, 25, 35, 45]
Sum of even numbers: 150
Sum of odd numbers: 120
```

Explanation of Improvements:

This refactored code calculates the sum of odd and even numbers from the numbers list using a single loop and conditional statements for clarity and efficiency.